LAB 7: FUZZING FOR INPUT VALIDATION BUGS

Student	Nidhi Poojamrutha	**SRN**	PES1UG23CS390
Experiment	Water Utility Billing - Input Validation	**Date**	13 November 2026

1. Objective

To identify input-validation defects in a Python water utility billing program using Hypothesis property-based fuzz testing, correct the defects through defensive input handling, and verify the updated program with Pytest.

2. Software and files

Python 3.11, Hypothesis 6.170.15, Pytest 9.2.0 and Visual Studio Code on macOS. The experiment uses **processor.py** and **test_processor.py** from the assigned Lab 7 repository.

3. Procedure

The original program was executed with valid sample values. Three Hypothesis tests were then completed to exercise the meter ID sanitizer, consumption-reading parser and billing-summary formatter with arbitrary Unicode strings and null inputs. The failing test results were reviewed, the processing functions were updated, and the application and test suite were run again.

4. Initial application execution

Inputs: **!MTR@123** (meter ID), **10,25,30** (consumption readings) and **Alice:50:North** (billing record). The original application processed these well-formed inputs correctly.

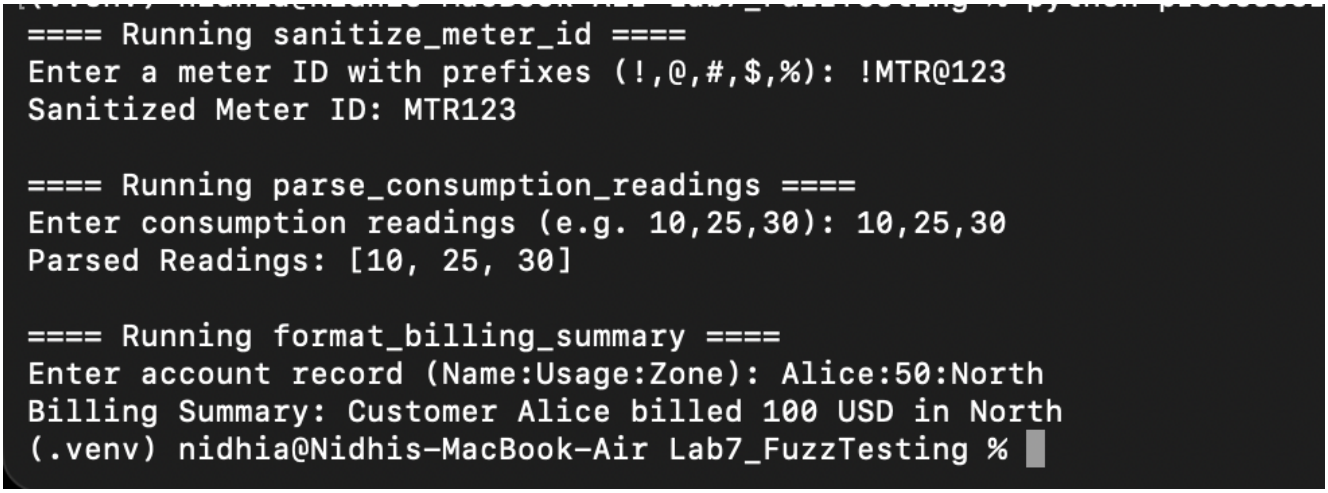


Figure 1. Initial execution of processor.py with valid input values.

5. Fuzz-testing results before correction

The Hypothesis tests exposed problems that did not appear during the normal execution. All three property-based tests failed before changes were made to the application.

Test function	Observed outcome
test_sanitize_meter_id_no_crash	Failed: NoneType has no attribute 'strip'.
test_parse_consumption_readings_safe	Failed: Hypothesis reported 2 distinct failures.
test_format_billing_summary_safe	Failed: Hypothesis reported 3 distinct failures.

Recorded test summary: 3 failed in 3.38 s. The test summary above was supplied as terminal text; a screenshot of the failed run was not supplied.

6. Corrections implemented

Meter ID sanitization: A type check was introduced before applying string operations; non-string input returns an empty string. Existing prefix removal and uppercase conversion were retained.

Consumption parsing: Non-string input returns an empty list. Blank comma-separated elements are skipped, valid integers are retained, and malformed elements do not cause an uncaught ValueError.

Billing summary: The record is checked for the expected three colon-separated fields, valid text values and an integer quantity. Invalid records return a controlled error message rather than raising an exception; negative units are rejected.

7. Execution after correction

The corrected program was run with the same sample inputs to confirm that handling malformed data had not changed the expected results for well-formed records.

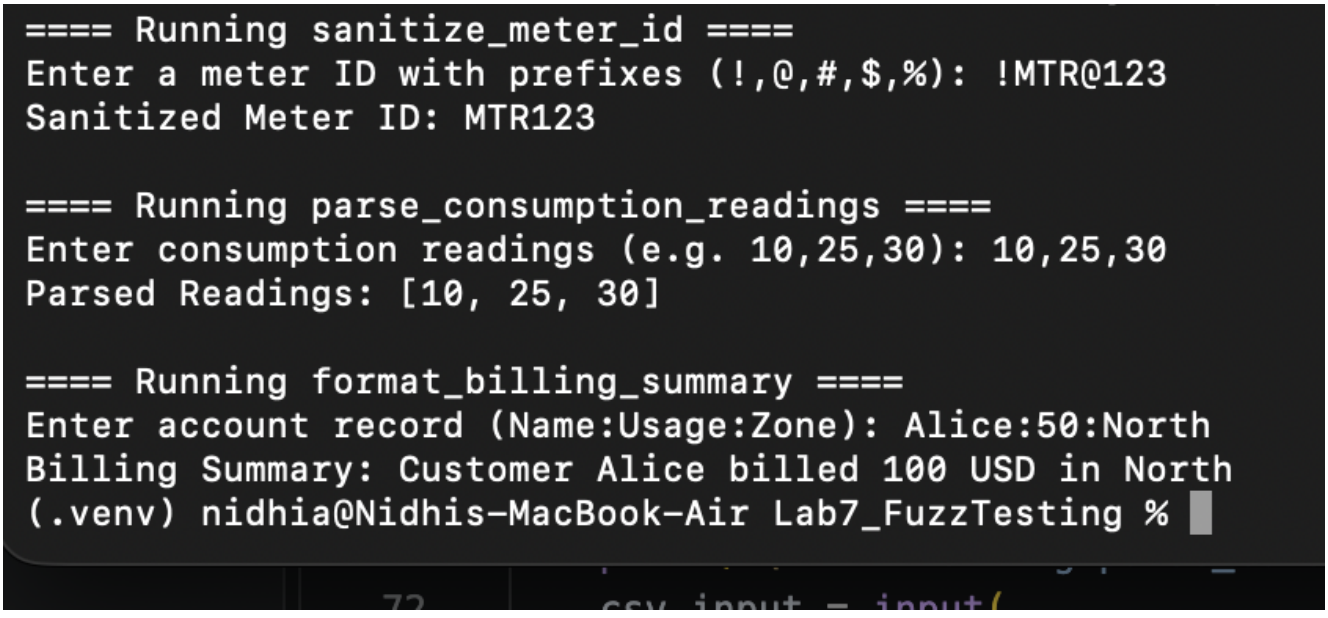


Figure 2. Corrected application successfully processes the sample billing inputs.

8. Final verification using Pytest

After updating processor.py, the Hypothesis test suite was run again using the command below.

```
python -m pytest test_processor.py -v
```

```
Billing Summary: Customer Alice billed 100 USD in North
[(.venv) nidhia@Nidhis-MacBook-Air Lab7_FuzzTesting % python -m pytest test_proce]
ssor.py -v
============================= test session starts ==============================
platform darwin -- Python 3.11.0, pytest-9.2.0, pluggy-1.7.0 -- /Users/nidhia/De
sktop/.venv/bin/python
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: /Users/nidhia/Desktop/Lab7_FuzzTesting
plugins: hypothesis-6.170.15
collected 3 items

test_processor.py::test_sanitize_meter_id_no_crash PASSED                [ 33%]
test_processor.py::test_parse_consumption_readings_safe PASSED           [ 66%]
test_processor.py::test_format_billing_summary_safe PASSED               [100%]

============================== 3 passed in 0.26s ===============================
(.venv) nidhia@Nidhis-MacBook-Air Lab7_FuzzTesting %
```

Figure 3. Final Pytest execution: all three property-based tests passed.

9. Result

The initial property-based tests revealed input-validation failures in all three processing functions. Following defensive changes to the application, the same three tests completed successfully. The final run reported **3 passed in 0.26 s**.

10. Conclusion

This experiment demonstrated how property-based fuzz testing identifies failures outside a program's normal input path. Type checking, validation of delimiters and numeric fields, and explicit handling of malformed values improved the robustness of the water utility billing processor. The reported passing tests verify the non-crashing behavior checked by the current suite; they do not establish correctness for every possible business-rule scenario.

Submission files

1. Lab 7 report (PDF) 2. Corrected processor.py 3. Updated test_processor.py